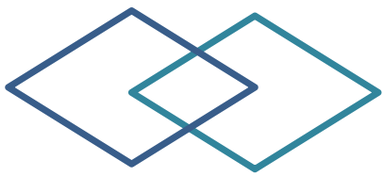



# 第4章

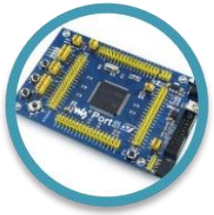
# STM32最小系统与嵌入式C语言



# 本章知识与能力要求

- 掌握STM32最小系统设计；
- 理解和掌握嵌入式C语言的程序结构；
- 掌握嵌入式C语言的数据类型、const、volatile和extern等关键字的使用；
- 掌握嵌入式C语言的条件编译；
- 掌握嵌入式C语言指针的应用；
- 学会分析HAL库文件源码。

# 第4章 STM32最小系统与嵌入式C语言



4.1 STM32最小系统




4.2 嵌入式C语言



4.3 HAL库文件源码分析

# 4.1 STM32最小系统

用最少的元件组成单片机可以工作的系统

01 4.1.1 电源电路

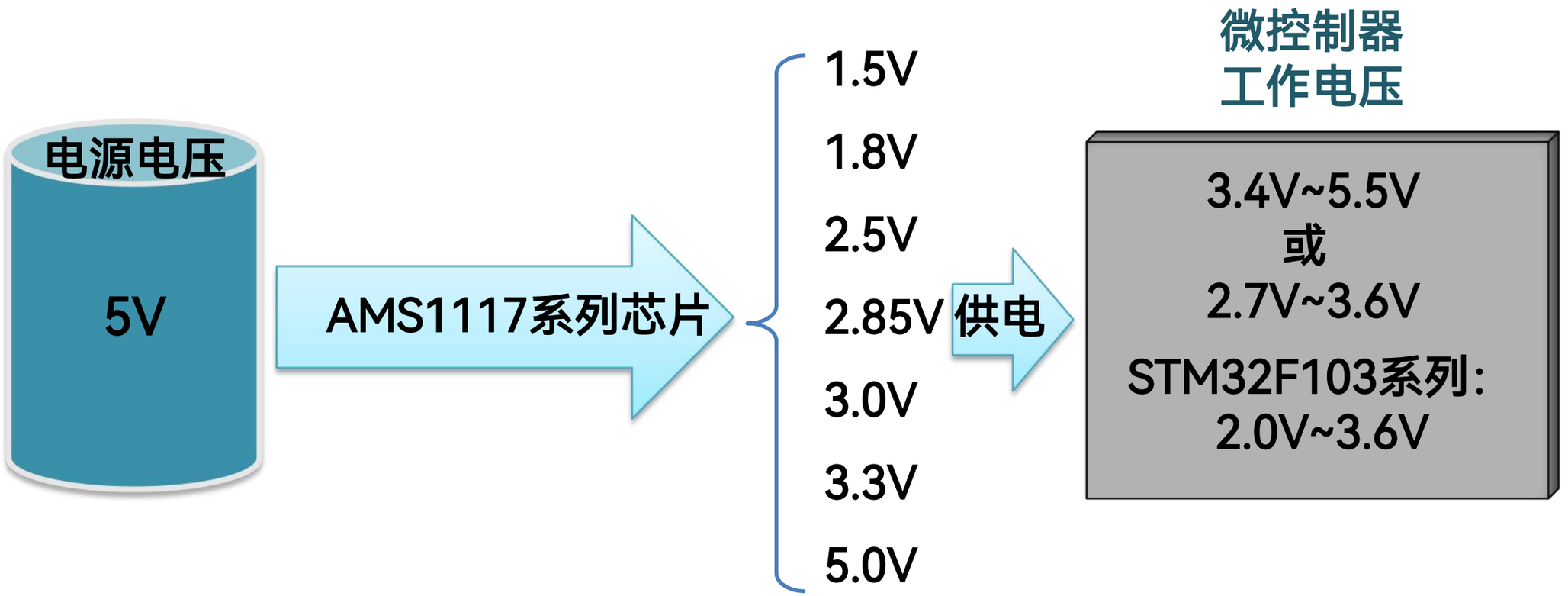
02 4.1.2 时钟电路（晶振电路）

03 4.1.3 复位电路

04 4.1.4 调试和下载电路

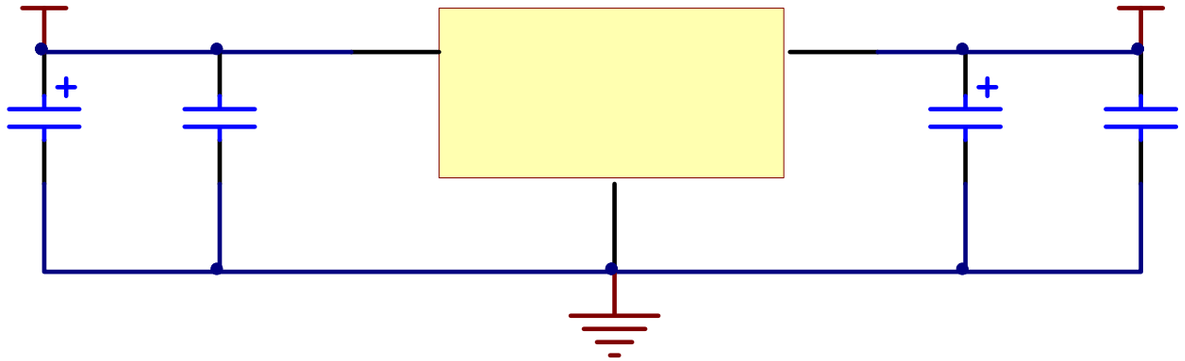
# 4.1.1 电源电路

## 为系统提供稳定可靠的供电



# 4.1.1 电源电路

## AMS1117-3.3v降压电路图



- C1、C3为输入电容，作用是防止断电后出现电压倒置
- C2、C4为输出滤波电容，作用是抑制自激振荡和稳定输出电压

电压转换电路提供3.3V电压供STM32主电源$V_{DD}$使用

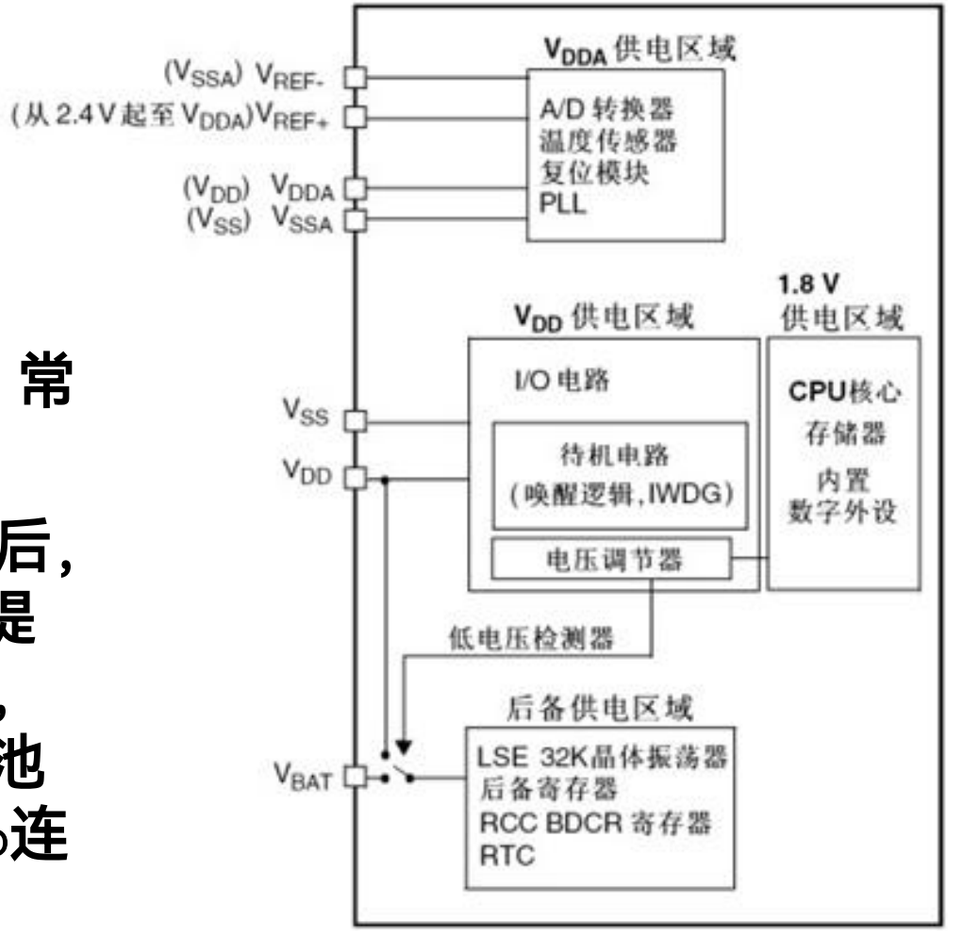
# 4.1.1 电源电路

STM32内部包含一个电压调节器，将外部3.3V电压转换为1.8V提供给Cortex-M3内核、内存以及外设使用。

内部有两个可选电源选项：

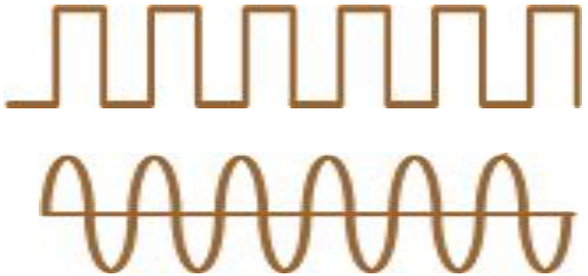
- ADC模块所需的参考电压$V_{REF+}$和$V_{REF-}$。常采用独立的供电电源。。
- 备用电池电源VBAT。当主电源$V_{DD}$掉电后，通过$V_{BAT}$引脚为实时时钟和备份寄存器提供电源。通常选择备用电池作为该电源，电压范围为1.8~3.6V。若不使用备用电池供电，则$V_{BAT}$引脚必须与主电源引脚$V_{DD}$连接。

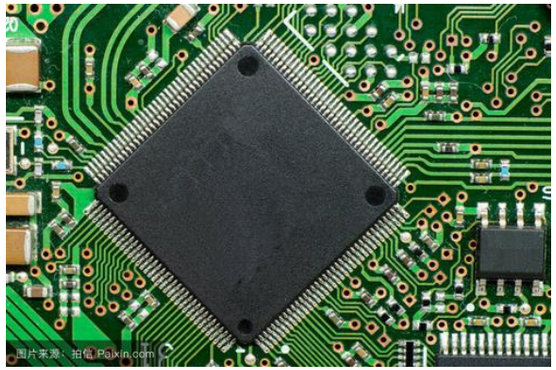
STM32内部电源框图



# 4.1.2 时钟电路(晶振电路)







时钟系统产生供系统正常工作的稳定的脉冲信号.

单片机的一切指令的执行都依赖于晶振提供的时钟频率，时钟频率越高，单片机运行速度越快，功耗也就越大。

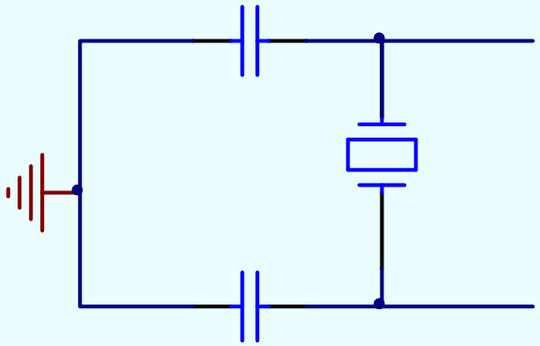
# 4.1.2 时钟电路(晶振电路)

晶振

- 有源晶振(振荡器)
  - 内部包含晶体管和阻容元件
  - 一个完整的谐振振荡器
  - 只需要给它供电
  - 不需要外接其他元件
  - 通常有四个引脚：VCC、GND、晶振输出引脚和一个悬空的引脚。
- 无源晶振(晶体)
  - 只有2个引脚的无极性元件
  - 需要借助外部电路起振
  - 精度上差一些
  - 价格便宜

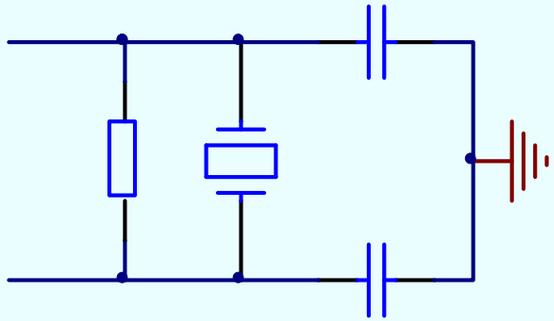
# 4.1.2 时钟电路(晶振电路)

01



使用低速晶振电路作为系统低速外部时钟源，频率为32.768kHz，主要用作RTC的时钟源，用于精准计时定时电路、万年历等。

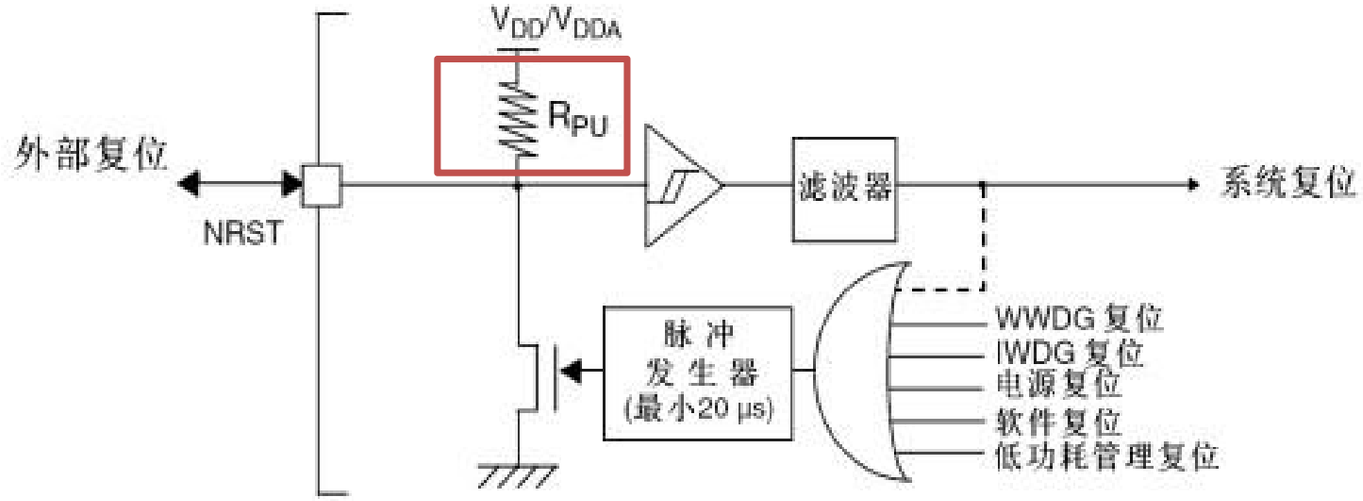
02



使用外部高速晶振电路作为系统时钟源，一般称为高速外部时钟信号(HSE）。一般选择8MHz，方便倍频。

# 4.1.3 复位电路

复位电路将系统进行复位




STM32内部复位电路

STM32芯片复位管脚持续为低电平时复位，STM32的NRST引脚在内部已经连接了一个上拉电阻，数据手册建议复位电路需外接一个对地电容，如果认为这个上拉电阻较小，用户也可以在复位电路中外接一个上拉电阻

# 4.1.3 复位电路

## 上电复位：

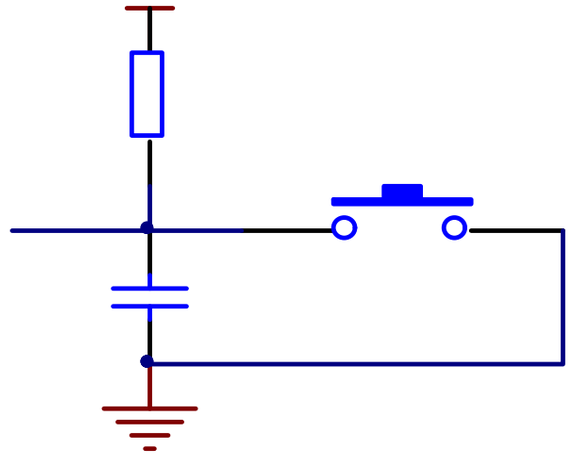
在上电瞬间，由于电容来两端电压不能突变，RESET出现短暂低电平，芯片自动复位，之后电容充电，充电时间由电阻和电容共同决定：

$t=1.1RC=1.1\times10k\Omega\times0.1\mu F=1.1ms$

上电后约1ms左右系统完成复位，之后，单片机开始正常工作。

## 手动复位：

按键S1按下时，RESET和地连通，产生低电平，实现复位。



# 4.1.4 调试和下载电路

**作用**

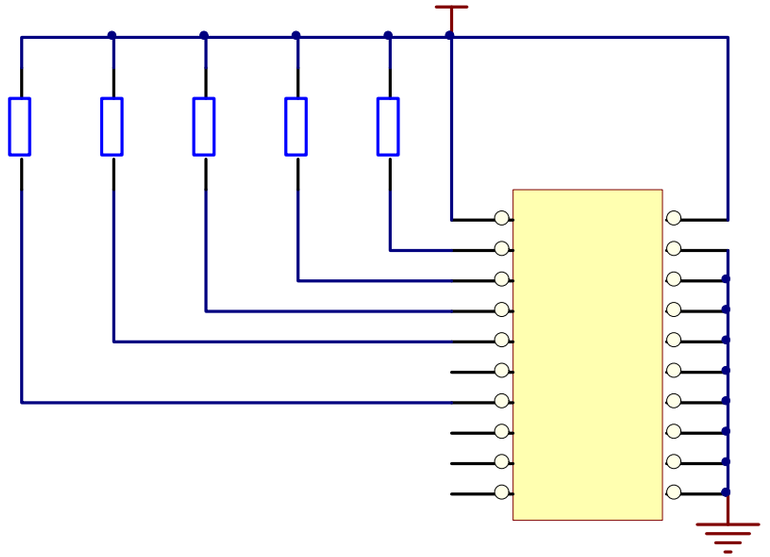
将编译好的程序下载到单片机中运行以及进行在线调试

STM32F103系列微控制器内核集成了SWD/JTAG调试端口(缩写为SWJ-DP)

- JTAG调试接口(5引脚的JTAG-DP)
- SWD(Serial Wire Debug，串行单总线调试）（2引脚的SW-DP)

## STM32调试端口引脚功能

| SWJ-DP引脚名称 | JTAG-DP | SW-DP | 引脚号 |
|---|---|---|---|
| JTMS/SWDIO | 输入：JTAG模式选择 | 输入输出：串行数据输入输出 | PA13 |
| JTCK/SWCLK | 输入：JTAG时钟 | 输入：串行时钟 | PA14 |
| JTDI | 输入：JTAG数据输入 | | PA15 |
| JTDO/TRACESWO | 输出：JTAG数据输出 | 异步跟踪 | PB3 |
| JNTRST | 输入：JTAG模块复位 | | PB4 |



# 4.1.4 调试和下载电路



Cortex-M3内核有三种启动方式，可以通过BOOT0和BOOT1的电平组合进行选择。

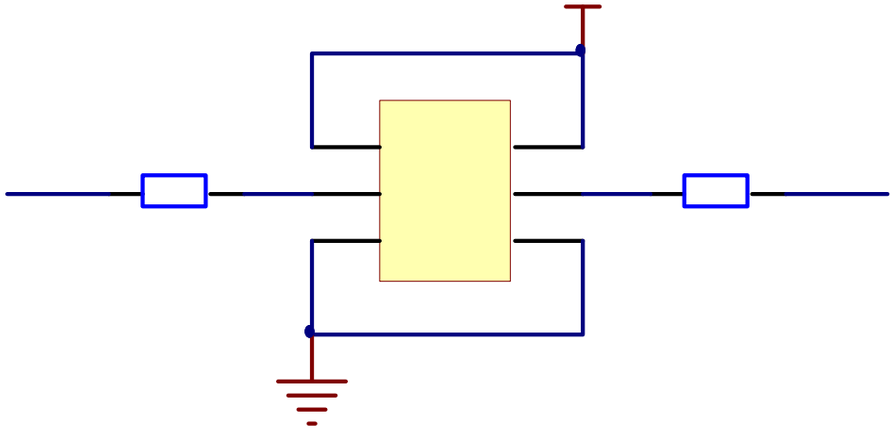
| 启动模式选择引脚 | | 启动模式对应的存储介质 | 说明 |
|---|---|---|---|
| BOOT0 | BOOT1 | | |
| 0 | x | 闪存存储器，即用户闪存Flash | 闪存存储器被选为启动区域 |
| 1 | 0 | 系统存储器，即系统Flash | 系统存储器被选为启动区域 |
| 1 | 1 | 内置SRAM | 内置SRAM被选为启动区域 |

在STM32F103系列微控制器中通过设置BOOT[1:0]引脚电平的高低选择三种不同启动模式，从而将STM32F103微控制器的存储空间起始地址0x00000000映射到不同存储区域的起始地址。

# 4.1.4 调试和下载电路



- 保持BOOT1为“0”，当BOOT0=1时，STM32F103系列微控制器从系统存储器的起始地址0x1FFFF000开始启动；
- 当BOOT0=0时，STM32F103系列微控制器从用户Flash的起始地址0x80000000开始启动运行代码，这是STM32最常用的启动方式。
- 若要从片内SRAM起始地址0x20000000启动，则BOOT0、BOOT1均设为“1”。



# 4.2 嵌入式C语言


4.2.1 STM32的数据类型

4.2.2 const 关键字

4.2.3 static 关键字

4.2.4 volatile 关键字

4.2.5 extern 关键字

4.2.6 struct结构体

4.2.7 enum

4.2.8 typedef

4.2.9 #define

4.2.10 #ifdef、#if条件编译

4.2.11 指针

4.2.12 回调函数

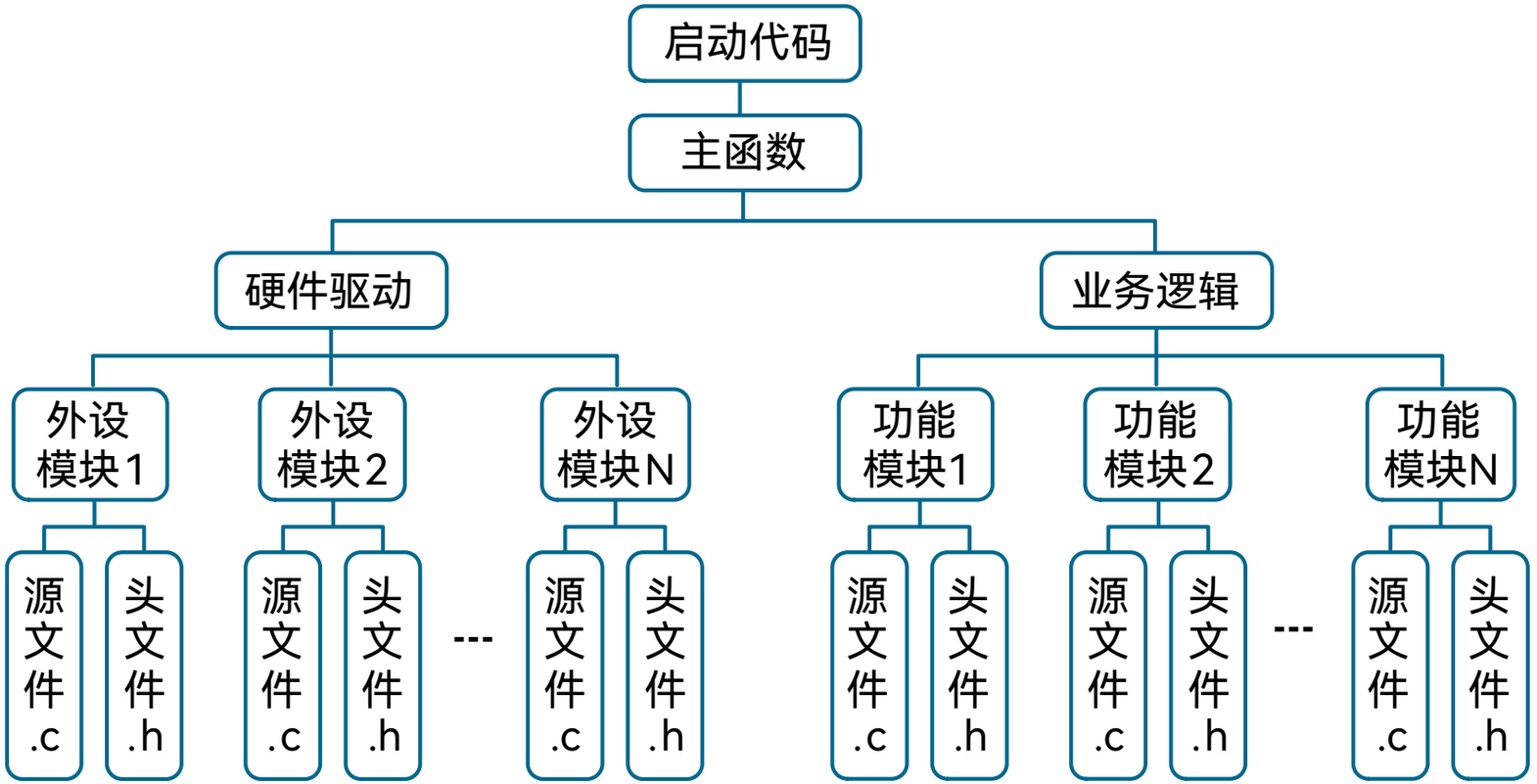
# 4.2 嵌入式C语言

嵌入式开发中既有底层硬件的开发又涉及上层应用的开发，即涉及系统的硬件和软件，C语言既具有汇编语言操作底层的优势，又具有高级语言功能性强的特点。

1. 程序总是从main函数开始执行；
2. 函数是C语言的基本结构；
3. 函数由两部分组成：函数说明部分和函数体。
4. 一个C语言程序包含若干个源程序文件（.c文件）和头文件(.h文件）。
5. 采用外设功能模块化设计方法，一个外设功能模块包括一个源文件(.c文件）和一个头文件(.h文件）。

# 4.2 嵌入式C语言

**嵌入式系统开发多采用模块化、层次化的设计思想，系统层次架构清晰，便于协同开发。**

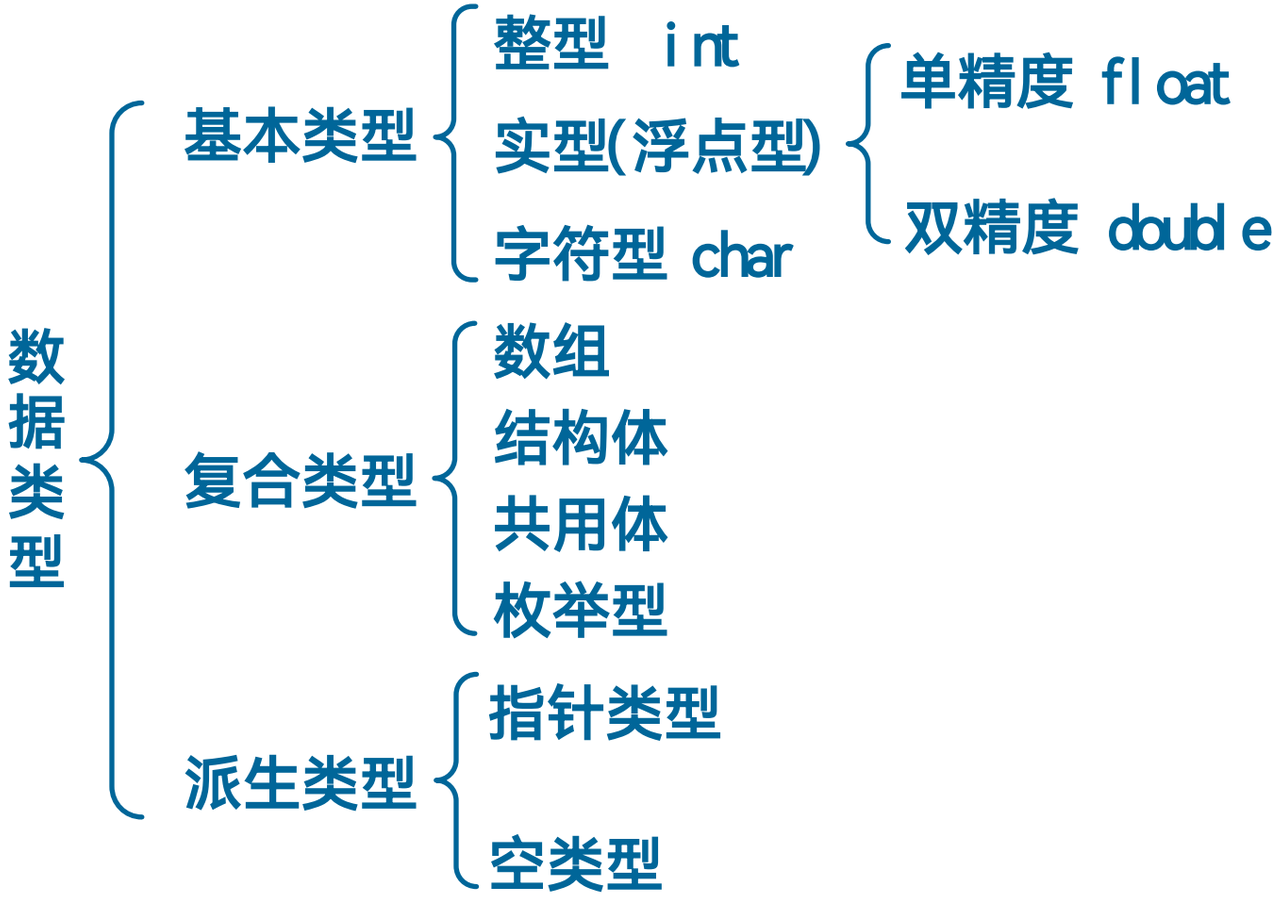



**嵌入式系统软件结构框图**

# 4.2.1 STM32的数据类型

数据是嵌入式C语言的基本操作对象。

数据类型是指数据在计算机内存中的存储方式。




嵌入式C语言的数据类型

# 4.2.1 STM32的数据类型

不同CPU所定义的数据类型的长度不一致，ST公司为开发人员提供了基于C语言的标准外设库，标准外设库中定义的数据类型的长度如下表所示。

| C语言的数据类型 | STM32对应的数据类型 | 说明 |
| --- | --- | --- |
| unsigned char | uint8_t | 8位无符号数据(0 ~ 255) |
| unsigned short int | uint16_t | 16位无符号数据(0 ~ 65535) |
| unsigned int | uint32_t | 32位无符号数据(0 ~ $2^{32}$-1) |
| unsigned long long | uint64_t | 64位无符号数据(0 ~ $2^{64}$-1) |
| signed char | int8_t | 8位有符号数据(-128 ~ +127) |
| signed short int | int16_t | 16位有符号数据(-32768 ~ +32767) |
| signed int | int32_t | 32位有符号数据(-$2^{31}$ ~ $2^{31}$-1) |
| signed long long | int64_t | 64位有符号数据(-$2^{63}$ ~ $2^{63}$-1) |

# 4.2.1 STM32的数据类型

v3.5.0版本已不再使用旧的数据类型，为了兼容以前的版本，stm32f10x.h头文件还对标准外设库之前版本所使用的数据类型进行了说明。

```
typedef uint32_t  u32;
typedef uint16_t u16;
typedef uint8_t  u8;

typedef __IO int32_t  vs32;
typedef __IO int16_t  vs16;
typedef __IO int8_t   vs8;

typedef __I int32_t vsc32;  /*!< Read Only */
typedef __I int16_t vsc16;  /*!< Read Only */
typedef __I int8_t vsc8;   /*!< Read Only */
```

STM32开发中同一数据类型有多种表示方式。

例

无符号8位整型数据有unsigned char，uint8_t，u8三种表示方式。

最新的v3.5.0版本采用标准的C99标准，即uint8_t方式。

# 4.2.1 STM32的数据类型

__I、__O以及__IO为IO类型限定词，内核头文件core_cm3.h定义了标准外设库所使用的IO类型限定词。

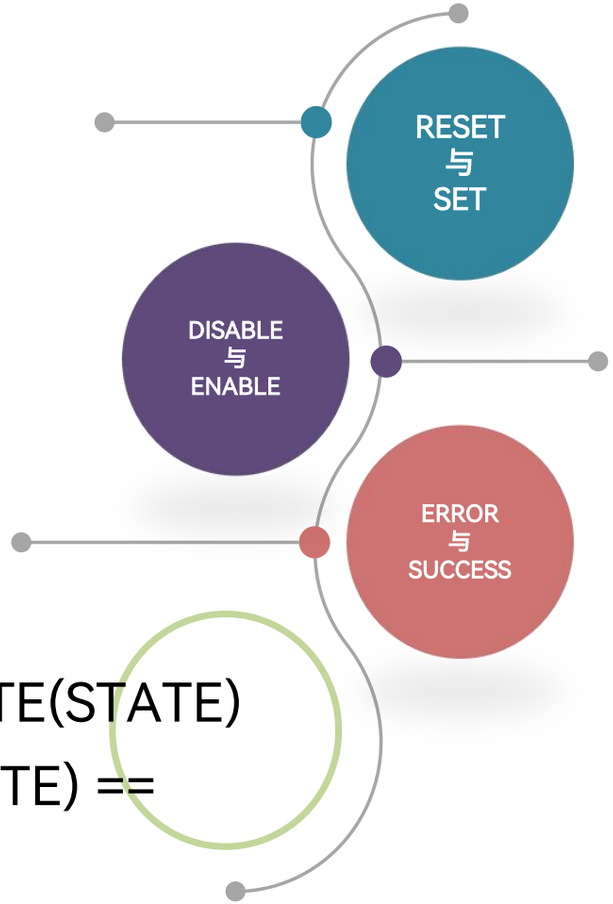
| IO类型限定词 | 类型 | 说明 |
| --- | --- | --- |
| __I | volatile const | 只读操作 |
| __O | volatile | 只写操作 |
| __IO | Volatile | 读和写操作 |

数据类型和IO类型限定词结合在一起，在标准外设库中常用来定义寄存器和结构体变量：

```
1000
1001  typedef struct
1002  {
1003    __IO uint32_t CRL;
1004    __IO uint32_t CRH;
1005    __IO uint32_t IDR;
1006    __IO uint32_t ODR;
1007    __IO uint32_t BSRR;
1008    __IO uint32_t BRR;
1009    __IO uint32_t LCKR;
1010  } GPIO_TypeDef;
```

# 4.2.1 STM32的数据类型

**stm32f10x.h头文件中常用的布尔类型的变量的定义**




```
typedef enum
{   DISABLE = 0,
    ENABLE
= !DISABLE
} FunctionalState;
```

```
#define IS_FUNCTIONAL_STATE(STATE)
(((STATE) == DISABLE) || ((STATE) ==
ENABLE))
```

```
typedef enum
{   RESET = 0,
    SET = !RESET
} FlagStatus, ITStatus;
```

```
typedef enum
{   ERROR = 0,
    SUCCESS = !ERROR
} ErrorStatus;
```

# 4.2.2 const关键字




## 作用

用于定义只读的变量，其值在编译时不能被改变

## 要求

const关键词修饰的变量在声明时必须初始化

## 目的

为了在编译时防止变量的值被误修改，提高程序的安全性和可靠性。

## 属性

在C标准中，const定义的变量是全局的。

## 格式

const 常量类型 常量名=常量表达式;

# 4.2.3 static关键词

**作用**

修饰变量或函数。修饰后的变量称为静态变量。

**操作**

在全局变量之前加上关键字static，则该全局变量被定义成为一个静态全局变量。

**目的**

作用范围只在定义该变量的源文件内有效，其他源文件不能引用该全局变量，避免了在其他源文件中因引用相同名字的变量而引发错误，有利于模块化程序设计。

# 4.2.3 static关键词

**static编程要点1：**

**模块化的程序设计中，用static声明一个函数，则该函数只能被该模块内的其它函数调用。**

```
#include "stm32f1xx_hal.h"
static void DMA_SetConfig(DMA_HandleTypeDef *hdma,
uint32_t SrcAddress, uint32_t DstAddress, uint32_t
DataLength);
… …
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef
*hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t
DataLength)
{
  HAL_StatusTypeDef status = HAL_OK;
… …
  if(HAL_DMA_STATE_READY == hdma->State)
  {
DMA_SetConfig(hdma, SrcAddress, DstAddress, DataLength);
… …
}
… …
}
```

**解析：** DMA_SetConfig()函数只能被stm32f1xx_hal_dma.c的其它函数调用，不能被其它模块的文件使用。

# 4.2.3 static关键词

**static编程要点2：**

static除了用于静态全局变量，还用于定义静态局部变量，保证静态局部变量在调用过程中不被重新初始化。

```
void fun_count( )
{
    static count_num = 0;
        // 声明一个静态局部变量，count_num用作计数器，初值为0。
    count_num ++;
    printf("%d\n", count_num);
}
int main(void)
{
    int i=0;
    for (i = 0;i <= 5;i++)
    {
      fun_count( );
    }
    return 0;
}
```

解析：在main函数中每调用一次fun_count( )函数，则静态局部变量count_num加1，而不是每次都被初始化为初值0。

# 4.2.4 volatile关键字

**使用方式**

一个类型修饰符，“易变的”。

volatile char i;

使用volatile关键字定义了一个字符型的变量i，指出i是随时可能发生变化的，每次使用的时候都必须从i的地址中读取。

**作用**

使用volatile就是不让编译器进行优化，即每次读取或者修改值的时候，都必须重新从内存中读取或者修改，而不是使用保存在寄存器里的备份。

**应用场合**

- 中断服务程序中修改的供其他程序检测的变量需要使用volatile;
- 多任务环境下各任务间共享的标志应添加volatile;
- 存储器映射的硬件寄存器通常也要加volatile进行说明。

# 4.2.5 extern关键词

使用extern是一个声明而不是重新定义

作用

指明此函数或变量的定义在别的文件中，提示编译器遇到此函数或变量时去其他模块中寻找其定义。

```
extern  int  a;
```

声明变量a，而不是在定义变量a，并未为a分配内存空间

```
extern  int  funA( );
```

声明函数funA( )，此函数已在其他文件中定义。

extern "C"进行链接指定，告知编译器这是采用C语言定义的函数

# 4.2.5 extern关键词

```
#ifdef __cplusplus
extern "C"{
#endif
……
#ifdef __cplusplus
}
#endif
```

**解析：** 如果定义了__cplusplus(C++编译器中自定义的宏），则执行extern“C”{语句。C++支持函数重载，而C语言不支持函数重载，在C++环境下使用C函数会出现链接时找不到对应函数的情况，这时需要使用extern “C”进行链接指定，告知编译器使用C语言的命名规则来处理函数。

当函数有可能被C语言或C++使用时，将函数声明放在extern “C”中以免出现编译错误。

```
#ifdef __cplusplus
extern "C"{
#endif
//函数声明
#ifdef __cplusplus
}
#endif
```

# 4.2.5 extern关键词

为保证全局变量和功能函数的使用，extern一般用在.h头文件中对某个模块提供给其它模块调用的外部函数及变量进行声明，实际编程中只需要将该.h头文件包含进该模块对应的.c文件，即在该模块的.c文件中加入代码#include "xxx.h"。

ADCx.h

```
//声明全局变量
extern  uint16_t  ADC_ConvertedValue; //存放ADC的转换结果
… …
//外部功能函数声明
extern  void  ADC_Init(void);// ADC初始化函数
… …
```

ADCx.c

```
#include  "ADCx.h"
… …
   //定义变量
static uint16_t  pwd=1;  //局部变量，仅作用于本函数
uint16_t  ADC_ConvertedValue; //全局变量
… …
void  ADC_Init(void)
{
   … …
}
… …
```

# 4.2.6 struct结构体

## 作用

struct用于定义结构体类型，其作用是将不同数据类型的数据组合在一起，构造出一个新的数据类型。

## 格式

struct一般用法为：

```
struct [结构体名]
{
    类型标识符  成员名1;
    类型标识符  成员名2;
        …… ……
}结构体变量;
```

## 举例

```
struct person
{   char name[8];
    int age;
    char sex[8];
    char address[20];
}person_liu;
```

# 4.2.6 struct结构体

结构体变量(如上例中的person_liu）可以不在定义结构体时定义，后续需要时再进行定义。

```
struct person
{
        char name[8];
        int age;
        char sex[8];
        char address[20];
};
struct  person  person_liu,
person_zhang;
```

定义格式

struct 结构体名 结构体变量;

使用这种定义方式可以很方便地同时定义多个结构体变量。

结构体变量的使用采用如下形式:

结构体变量名. 成员名

如: person_liu.age=35;

# 4.2.7 enum关键词

## enum枚举类型的用法:

```
enum 枚举名
{
    枚举成员1,
    枚举成员2,
     ……
}枚举变量;
```

用来将一个变量或对象的所有可能的值一一列出，变量取值只限于列举出来的值。

```
enum Weekdays
{
      Monday = 1,
      Tuesday,
      Wednesday,
      Thursday,
      Friday,
      Saturday,
      Sunday
}Mydays，Olddays;
```

枚举成员的值是常量，不是变量，不能被赋值，但可以将枚举值赋给枚举变量。

枚举类型具有自动编号的功能，第一个枚举成员其默认值为整型的0，后续枚举成员的值在前一个成员上加1。

可以自定义枚举成员的值，如果把第一个枚举元素的值定义为1，那么第二枚举成员的值就为2，以此类推，如上述例子中Friday的值为5。

# 4.2.8 typedef

作用：

- 用来为复杂的声明定义一个简单的别名，方便记忆

目的：

- 给变量起一个容易记且意义明确的新名字；
- 简化一些比较复杂的类型声明。

不是一个真正意义上的新类型

为已知的数据类型起一个简单的别名

## 用法一：typedef的基本应用

格式：

typedef 类型名 自定义的别名;

举例：

```
typedef signed char int8_t;
//给数据类型signed char起个别名int8_t
typedef signed int int32_t;
//给数据类型signed int起个别名int32_t
```

# 4.2.8 typedef

## 用法二：与结构体struct结合使用

STM32标准外设库中stm32f10x_gpio.h头文件中利用结构体别名GPIO_InitTypeDef

```
typedef struct
{
    uint16_t  GPIO_Pin;
    GPIOSpeed_TypeDef
GPIO_Speed;
    GPIOMode_TypeDef
GPIO_Mode;
```

使用typedef为这个新建的结构体起了一个新的名字叫GPIO_InitTypeDef

则可以使用GPIO_InitTypeDef定义一个变量GPIO_InitStructure，从而调用GPIO_Mode。

```
GPIO_InitTypeDef  GPIO_InitStructure;
GPIO_InitStructure.GPIO_Mode =
GPIO_Mode_Out_PP;
```

# 4.2.8 typedef

## 用法三：与enum结合使用

stm32f10x_gpio.h头文件中的代码。

```
typedef  enum
{
        GPIO_Speed_10MHz = 1,
        GPIO_Speed_2MHz,
        GPIO_Speed_50MHz
}GPIOSpeed_TypeDef;
```

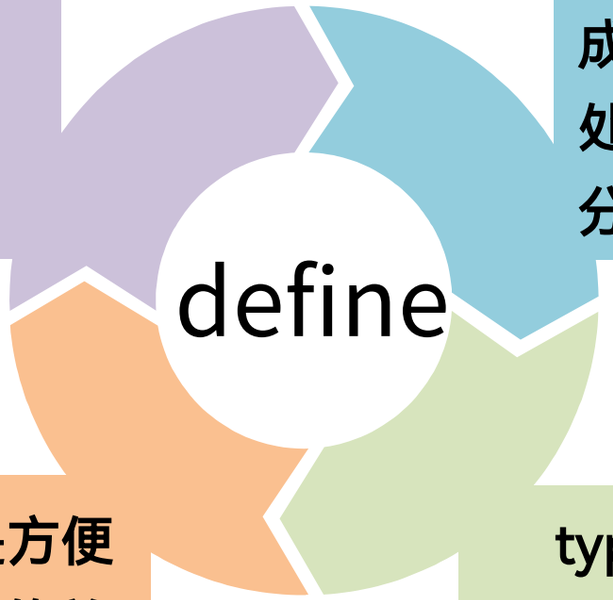
**解析1：** 利用typedef关键字将此枚举类型定义一个别名GPIOSpeed_TypeDef，这里省略了枚举类型的枚举名，只用typedef起了个别名。

**解析2：** enum枚举类型共三个成员，并将第一个枚举成员GPIO_Speed_10MHz赋值为1，enum枚举类型会将枚举成员的赋值在第一个枚举成员赋值的基础上加1，因此，GPIO_Speed_2MHz默认值为2。

# 4.2.9 #define

#define是C语言中的预处理命令，它用于宏定义，用来将一个标识符定义为一个字符串，该标识符称为宏名，被定义的字符串称为替换文本。

所谓预处理是指在编译之前所做的工作，由预处理程序负责完成，编译时，系统将自动引用预处理程序对源程序中的预处理部分进行处理。




采用宏定义的目的主要是方便程序编写，一般放在源文件的前面，称为预处理部分。

typedef与#define的区别：

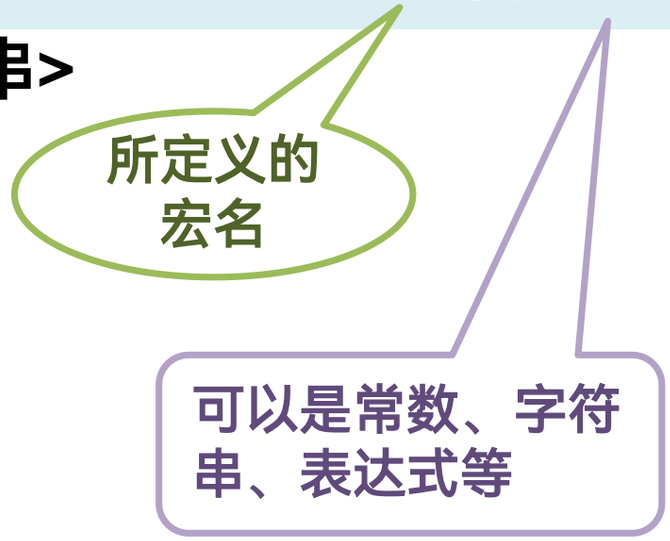
- typedef是在编译阶段处理的;
- #define是在预处理阶段处理的。

# 4.2.9 #define

## 用法一：无参数宏定义

**定义格式：**

#define <宏名> <字符串>




例1：#define UINT8_MAX 255

解析：定义宏名UINT8_MAX，代表255。

例2：#define _ _IO volatile;

解析：定义宏名_ _IO，表示volatile，以后程序中再需要用到volatile，就可以使用_ _IO。

例3：#define RCC_AHBPeriph_DMA1 ((uint32_t)0x00000001)

解析：定义RCC_AHBPeriph_DMA1宏名，代表32位的无符号数据0x00000001。

# 4.2.9 #define

## 用法一：无参数宏定义

标准外设库v3.5.0的stm32f10x_rcc.h文件中APB2_peripheral外设基地址的定义

```
]/** @defgroup APB2_peripheral
  * @{
  */

#define RCC_APB2Periph_AFIO              ((uint32_t)0x00000001)
#define RCC_APB2Periph_GPIOA             ((uint32_t)0x00000004)
#define RCC_APB2Periph_GPIOB             ((uint32_t)0x00000008)
#define RCC_APB2Periph_GPIOC             ((uint32_t)0x00000010)
#define RCC_APB2Periph_GPIOD             ((uint32_t)0x00000020)
#define RCC_APB2Periph_GPIOE             ((uint32_t)0x00000040)
#define RCC_APB2Periph_GPIOF             ((uint32_t)0x00000080)
#define RCC_APB2Periph_GPIOG             ((uint32_t)0x00000100)
#define RCC_APB2Periph_ADC1              ((uint32_t)0x00000200)
#define RCC_APB2Periph_ADC2              ((uint32_t)0x00000400)
#define RCC_APB2Periph_TIM1              ((uint32_t)0x00000800)
#define RCC_APB2Periph_SPI1              ((uint32_t)0x00001000)
#define RCC_APB2Periph_TIM8              ((uint32_t)0x00002000)
#define RCC_APB2Periph_USART1            ((uint32_t)0x00004000)
#define RCC_APB2Periph_ADC3              ((uint32_t)0x00008000)
#define RCC_APB2Periph_TIM15             ((uint32_t)0x00010000)
#define RCC_APB2Periph_TIM16             ((uint32_t)0x00020000)
#define RCC_APB2Periph_TIM17             ((uint32_t)0x00040000)
#define RCC_APB2Periph_TIM9              ((uint32_t)0x00080000)
#define RCC_APB2Periph_TIM10             ((uint32_t)0x00100000)
#define RCC_APB2Periph_TIM11             ((uint32_t)0x00200000)
```

APB2_peripheral各外设基地址的定义

# 4.2.9 #define

## 用法二：带参数的宏定义

**定义格式：**

#define<宏名>(参数1，参数2，…参数n）<替换列表>

例4：#define SUM(x,y) (x+y)

……

a = SUM(2,2);

解析：将SUM(x,y)定义为x+y，预编译时会将SUM(x,y)替换为x+y，a的结果是4

例5：#define IS_GPIO_SPEED(SPEED) (((SPEED) == GPIO_Speed_10MHz) || ((SPEED)
== GPIO_Speed_2MHz) || ((SPEED) == GPIO_Speed_50MHz))

解析：使用宏定义#define将IS_GPIO_SPEED(SPEED)替换为GPIO_Speed_10MHz或者GPIO_Speed_2MHz或者GPIO_Speed_50MHz。

# 4.2.9 #define

## 用法二：带参数的宏定义

STM32标准外设库stm32f10x.h头文件中的代码：

```
#define  IS_FUNCTIONAL_STATE(STATE) (((STATE) == DISABLE) || ((STATE) == ENABLE))
```

解析：该函数为外设时钟使能函数，第一个参数为要使能的外设，第二个参数为是否使能。

在stm32f10x_rcc.c源文件中函数RCC_APB2PeriphClockCmd()使用了IS_FUNCTIONAL_STATE(STATE)这个宏

RCC_APB2PeriphClockCmd()源码如下

```
void RCC_APB2PeriphClockCmd(uint32_t
  RCC_APB2Periph, FunctionalState NewState)
  {
        /* Check the parameters */

assert_param(IS_RCC_APB2_PERIPH(RCC_APB2Perip
h));
 assert_param(IS_FUNCTIONAL_STATE(NewState));
 if (NewState != DISABLE)
  {
    RCC->APB2ENR |= RCC_APB2Periph;
  }
 else
  {
    RCC->APB2ENR &= ~RCC_APB2Periph;
  }
```

# 4.2.10 #ifdef、#ifndef、#else、#if条件编

只有满足一定条件才进行编译，一般用在头文件或文件开头部分

嵌入式C语言常使用条件编译，通过条件判断来确定是否对某段源程序进行编译。

条件编译的用途：可以用源程序产生不同版本。

嵌入式C语言常用的条件编译命令

| 条件编译 | 说明 |
| --- | --- |
| #define | 宏定义 |
| #undef | 撤销已定义的宏名 |
| #if | 条件编译命令，如果# if后面的表达式为true，则执行语句 |
| #ifdef | 判断某个宏是否被定义，若被定义，则执行语句 |
| #ifndef | 判断某个宏是否未被定义，若未被定义，则执行语句，与#ifdef相反 |
| #elif | #else指令用于#if指令之后，当#if指令的条件不为真时，就编译#else后面的代码，elif相当于else if |
| #endif | 条件编译的结束命令，用在#if、#ifdef、#ifndef之后 |

# 4.2.10 #ifdef、#ifndef、#else、#if条件编

形式一：

```
#ifdef  标识符
     程序段1
#else
    程序段2
#endif
```

功能：当指定的标识符已被#define定义过，则只编译程序段1，否则编译程序段2。

```
#ifdef  IN_XXX
   #define  XXX_EXT
#else
   #define  XXX_EXT  extern
#endif
…….
XXX_EXT  volatile  u16  Name;
```

如果定义了IN_XXX，则定义XXX_EXT，否则定义XXX_EXT为extern。

# 4.2.10 #ifdef、#ifndef、#else、#if条件编

形式二：

```
#ifndef  标识符
     程序段1
#else
      程序段2
#endif
```

功能：当指定的标识符没有被#define定义过，则编译程序段1，否则编译程序段2。

标准外设库v3.5.0版本中的stm32f10x_rcc.h头文件中的源码

```
#ifndef STM32F10X_CL
  #define RCC_USBCLKSource_PLLCLK_1Div5
((uint8_t)0x00)
  #define RCC_USBCLKSource_PLLCLK_Div1
((uint8_t)0x01)
  #define IS_RCC_USBCLK_SOURCE(SOURCE) (((SOURCE)
  == RCC_USBCLKSource_PLLCLK_1Div5) ||\
  ((SOURCE) == RCC_USBCLKSource_PLLCLK_Div1))
#else
  #define RCC_OTGFSCLKSource_PLLVCO_Div3
((uint8_t)0x00)
  #define RCC_OTGFSCLKSource_PLLVCO_Div2
((uint8_t)0x01)
  #define IS_RCC_OTGFSCLK_SOURCE(SOURCE)
(((SOURCE) == RCC_OTGFSCLKSource_PLLVCO_Div3)||\
  ((SOURCE) == RCC_OTGFSCLKSource_PLLVCO_Div2))
```

# 4.2.10 #ifdef、#ifndef、#else、#if条件编

形式三：

```
#ifdef  标识符1(或表达式1)
    程序段1
#elif  标识符2(或表达式2)
    程序段2
#endif
```

功能：当定义了标识符1或表达式1，则编译程序段1；否则，如果定义了标识符2或表达式2，则编译程序段2。

标准外设库v3.5.0版本中的stm32f10x_rcc.h头文件中的源码。

```
#ifdef STM32F10X_CL
   /* PREDIV1 clock source (for STM32 connectivity line devices)
*/
   #define  RCC_PREDIV1_Source_HSE   ((uint32_t)0x00000000)
   #define  RCC_PREDIV1_Source_PLL2   ((uint32_t)0x00010000)
   #define IS_RCC_PREDIV1_SOURCE(SOURCE)
   (((SOURCE)==RCC_PREDIV1_Source_HSE) ||\
        ((SOURCE) == RCC_PREDIV1_Source_PLL2))
#elif defined (STM32F10X_LD_VL) || defined (STM32F10X_MD_VL)
||\
   defined (STM32F10X_HD_VL)
   #define  RCC_PREDIV1_Source_HSE   ((uint32_t)0x00000000)
   #define IS_RCC_PREDIV1_SOURCE(SOURCE) (((SOURCE) ==
   RCC_PREDIV1_Source_HSE))
#endif
```

# 4.2.11 指针

- **指针**

  用于存放地址的变量

指针理解1

◆ 有确定的数据类型

数据类型 *变量名;

如 int *p ;

特殊的：void * 类型代表通用指针

- **两个要素：**
  - ◆ 值：指的是某个对象的位置（即内存地址）；
  - ◆ 类型：是指对象所在位置上所存储数据的类型。

指针理解2

◆取地址运算符&指向一个变量的存储地址；

```
int  i;
int  *p;
p = &i;
```

//将int类型的指针p指向变量i的地址

# 4.2.11 指针

指针理解3

◆取值运算符*访问指针变量所指向地址单元的内容；

```
int i = 0;
int *p;
p = &i;
printf("%d \n", *p);
  //通过*p获取i的值用于输出
*p = 200;   //通过*p修改变量i的值
printf("%d \n", i);
  //输入i 的值，i=200
```

指针理解4

◆指针和数组

在数组中使用指针，可以通过移动指针，对数组中的元素进行搜索。

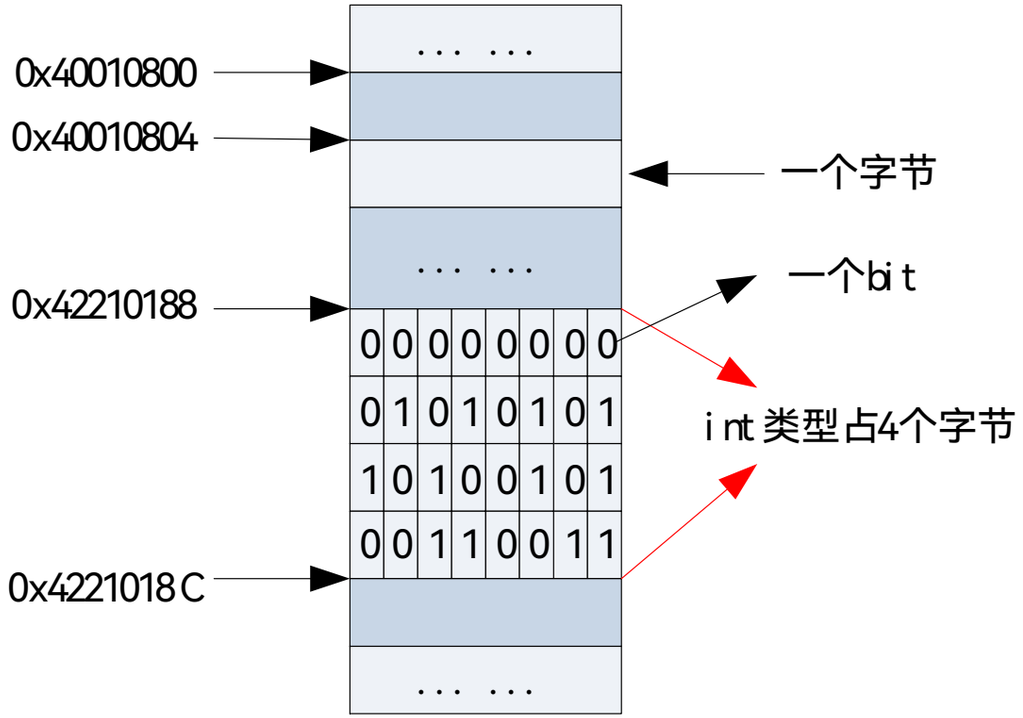
```
int Data_Array[] = {1,2,3,4,5,6};
    //定义一个数组
int *pr; //定义一个指针pr
pr = & Data_Array[0];
        //将指针指向数组的首地址
```

则，可以利用指针对数组的元素进行赋值

```
*pr=8;  //用指针给数组的第一个元素赋值
*(pr + 1)=9;  //用指针给数组的第二个元素赋值
*(pr + 2)=7;  //用指针给数组的第三个元素赋值
```

# 4.2.11 指针

32位微处理器中int类型的变量占4个字节的空间。




指针理解5

◆ 将指针从一种类型强制转换成另外一种类型。

pr=(int *) 0x42210188;

解析：0x42210188是一个32位的数据，表示的是存储空间中的一个地址，但是在程序中如果写成

pr=0x42210188;

只是将0x42210188这个32位的数据赋给pr，而使用强制类型转换(int *)，则是告知编译器这是一个整型数据所占内存空间的首地址，如果没有这个类型转换，则编译器会报错。*pr表示取这个指针所指向内存空间中存储的数据，如图，*pr=0x0055。

# 4.2.11 指针

函数指针的定义方式为：

函数返回值类型 (*指针变量名) (函数参数列表);

编译器会为函数分配一段连续的存储空间，其首地址就是这个函数名所定义的变量的地址

定义一个指针变量来存放该地址，这个指针变量就叫作函数指针变量，简称函数指针。

在程序中可以通过这个函数指针变量调用这个函数。

例如：

```
int f(int x, int *p)  ;//函数
……
int (* fp) (int, int *);
    //声明一个指针fp
fp = f;
    //将函数f赋给该指针
……
int y = 1;
int result = fp(3, &y);
    //用指针来调用f函数
```

# 4.2.11 指针

STM32标准外设库中，指针的类型为32位整型数据

STM32中片上外设都是挂接在不同的总线上的，stm32f10x.h文件中利用C语言的宏定义和指针来一步步地封装总线和外设的基地址。

## STM32标准外设库中的指针应用

通过宏定义为外设基地址0x40000000取一个宏名PERIPH_BASE，方便使用，stm32f10x.h中FLASH、SRAM和PERIPH(外设）的基地址定义如下：

```
#define FLASH_BASE ((uint32_t)0x08000000)
#define SRAM_BASE ((uint32_t)0x20000000)
#define PERIPH_BASE ((uint32_t)0x40000000)
```

STM32的总线有APB1、APB2和AHB总线组成，各自的总线基地址定义如下：

```
#define APB1PERIPH_BASE PERIPH_BASE
#define APB2PERIPH_BASE (PERIPH_BASE + 0x10000)
#define AHBPERIPH_BASE (PERIPH_BASE + 0x20000)
```

# 4.2.11 指针

stm32f10x.h文件中定义了这7组GPIO端口的基地址的宏。

```
#define GPIOA_BASE   (APB2PERIPH_BASE + 0x0800)
#define GPIOB_BASE   (APB2PERIPH_BASE + 0x0C00)
#define GPIOC_BASE   (APB2PERIPH_BASE + 0x1000)
#define GPIOD_BASE   (APB2PERIPH_BASE + 0x1400)
#define GPIOE_BASE  (APB2PERIPH_BASE + 0x1800)
#define GPIOF_BASE   (APB2PERIPH_BASE + 0x1C00)
```

ST公司的工程师采用了C语言结构体的形式封装了这些寄存器组：

```
#define GPIOA  ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB  ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC  ((GPIO_TypeDef *) GPIOC_BASE)
#define GPIOD  ((GPIO_TypeDef *) GPIOD_BASE)
#define GPIOE  ((GPI      Def *)
GP
G
G
```

结构体类型后面加上“*”号，表示是结构体类型的指针，(GPIO_TypeDef *)则是把GPIOA_BASE等的地址强制转换为GPIO_TypeDef结构体类型的指针，实际编程时可以直接用该宏名访问寄存器。

# 4.2.11 指针

GPIO_TypeDef的相关定义：

```
typedef struct
{
  __IO uint32_t CRL;
  __IO uint32_t CRH;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t BRR;
  __IO uint32_t LCKR;
} GPIO_TypeDef;
```

声明了名为GPIO_TypeDef的结构体类型，C语言中struct结构体中成员在内存中是按顺序存储的。

有了寄存器的地址，就可以使用指针进行读写操作了。

```
GPIOA_>BSRR | = (1 << 10) ;
    //等同于 *((int *) (0x 0x4001 0810) ) | = (1
<<10);
GPIOA->ODR =0xFF;
    //等同于 *((int *) (0x 0x4001 080C) ) = 0xFF;
```

ST工程师把对底层寄存器的操作进行了封装，以API的形式供开发人员调用，如STM32提供的库函数GPIO_SetBits()，用于设置某位引脚为高电平：

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t
GPIO_Pin)
{
    GPIOx->BSRR = GPIO_Pin;
}
```

库函数GPIO_SetBits()实质上还是对寄存器BSRR进行读写操作，只是进行了封装，开发人员只需调用此函数就可以实现对BSRR寄存器的操作。

# 4.2.12 回调函数

- 通过函数指针调用的函数。
- 操作系统中的某些函数常需要调用用户定义的函数来实现其功能，由于与常用的用户程序调用系统函数的调用方向相反，因此将这种调用称为“回调”（Callback），而被系统函数调用的函数称为“回调函数”。
- 回调函数是指通过调用其他函数反过来调用某个函数。

## STM32的HAL库stm32f1xx_hal_gpio.c源码

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}

__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    UNUSED(GPIO_Pin);
}
```

GPIO中断处理函数

调用回调函数

STM32的HAL库中GPIO引脚触发中断后的回调函数

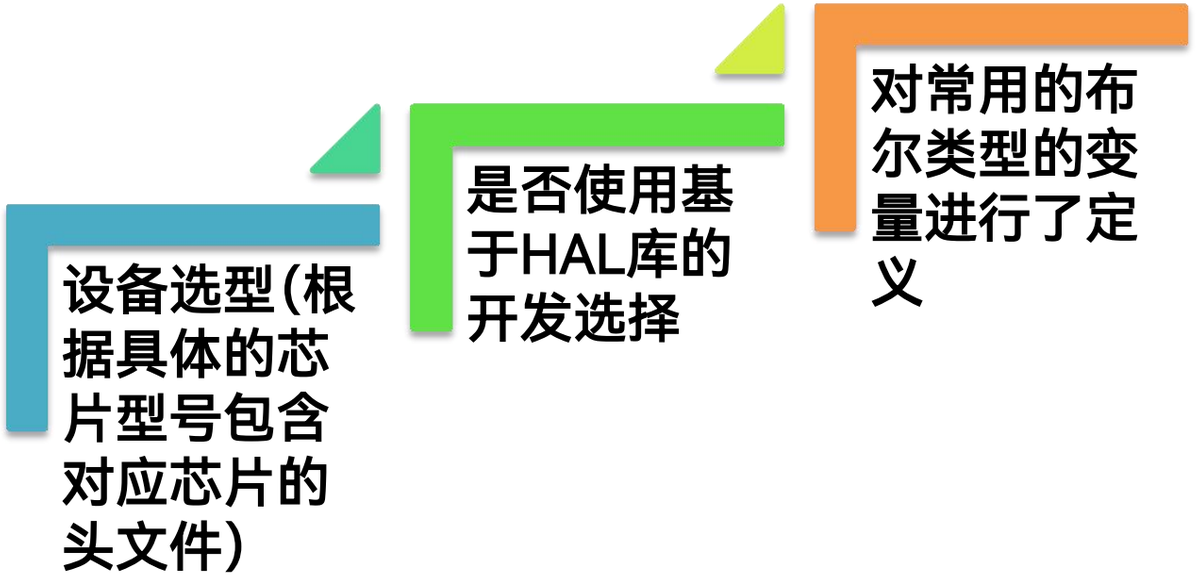
STM32的HAL库的回调函数的函数体需要用户自己编写，其实质是通过中断处理函数调用回调函数来实现中断服务功能

# 4.3 HAL库文件源码分析



# 4.3.1 stm32f1xx.h源码分析

stm32f1xx.h是HAL库中非常重要的头文件，是STM32F1xx系列MCU的顶层头文件，意味着凡是使用STM32F1xx系列的芯片，其应用程序都需要包含这个头文件。



# 4.3.1 stm32f1xx.h源码分析

```
#ifndef
__STM32F1XX_H
#define
__STM32F1XX_H

#ifdef __cplusplus
extern "C" {

#if !defined (STM32F1)
#def   STM32F1
```

解析：如果没有定义宏__STM32F1XX_H，则定义宏__STM32F1XX_H。这是个条件编译，目的是防止头文件被重复包含。

解析：这是一个条件编译，__cplusplus是C++中自定义的一个宏，plus是“+”的意思。这段代码的作用是如果定义了C++，则用extern"C"{" 和 " }"来告诉编译器:这是一个用C语言写成的库函数，请用C语言的方式来链接编译它们。主要是因为c++中存在重载，c++编译时会把函数参数一起编译，而c语言只编译函数名。

解析：
如果没定义STM32F1，则定义STM32F1。

# 4.3.1 stm32f1xx.h源码分析

```
#if !defined (STM32F100xB) && !defined (STM32F100xE) && !defined (STM32F101x6) && \
!defined (STM32F101xB) && !defined (STM32F101xE) && !defined (STM32F101xG) && !defined
(STM32F102x6) && !defined (STM32F102xB) && !defined (STM32F103x6) && \
!defined (STM32F103xB) && !defined (STM32F103xE) && !defined (STM32F103xG) && !defined
(STM32F105xC) && !defined (STM32F107xC)
  /* #define STM32F100xB  */
  /* #define STM32F100xE */
  /* #define STM32F101x6  */
  /* #define STM32F101xB  */
  /* #define STM32F101xE */
  /* #define STM32F101xG  */
  /* #define STM32F102x6 */
  /* #define STM32F102xB  */
  /* #define STM32F103x6  */
  /* #define STM32F103xB  */
  /* #define STM32F103xE */
  /* #define STM32F103xG  */
  /* #define STM32F105xC */
  /* #define STM32F107xC  */
#endif
```

解析：这段宏定义作用是编译器提示进行设备选型，如果没有发现任何STM32设备，则编译器在编译时会提示错误信息，如： #error "Please select first the target STM32F1xx device used in your application (in stm32f1xx.h file)"，这时就需要在这段代码中找到所使用的MCU，把相应系列的STM32芯片的注释去掉，就可以了，如使用的是STM32F103ZET6芯片，则需要把STM32F103xE这段代码前后的注释去掉。

# 4.3.1 stm32f1xx.h源码分析

```
#if !defined  (USE_HAL_DRIVER)
/**
 * @brief Comment the line below if you will not use the peripherals drivers.
   In this case, these drivers will not be included and the application code will be based on direct access to peripherals registers
   */
  /*#define USE_HAL_DRIVER */
```

解析：这段代码用于定义是否使用外设驱动库编写外设应用程序(这儿指HAL库或标准外设库，即调用API函数的开发方式），如果想使用基于寄存器的开发方式，即不使用ST提供的HAL库，则把下面的代码注释掉。

```
#define __STM32F1_CMSIS_VERSION_MAIN (0x04)
        /*!< [31:24] main version */
#define __STM32F1_CMSIS_VERSION_SUB1 (0x03)
        /*!< [23:16] sub1 version */
#define __STM32F1_CMSIS_VERSION_SUB2 (0x01)
         /*!< [15:8]  sub2 version */
#define __STM32F1_CMSIS_VERSION_RC (0x00)
         /*!< [7:0]  release candidate */
#define __STM32F1_CMSIS_VERSION ((__STM32F1_CMSIS_VERSION_MAIN << 24)\
|(__STM32F1_CMSIS_VERSION_SUB1 << 16)\
|(__STM32F1_CMSIS_VERSION_SUB2 << 8 )\
|(__STM32F1_CMSIS_VERSION_RC))
```

解析：CMSIS设备版本号的定义。

# 4.3.1 stm32f1xx.h源码分析

解析：stm32f1xx.h是STM32F1 MCU的顶层头文件，由于STM32F1xx系列的芯片特别多，ST针对每种具体芯片型号定义了一个特有的设备驱动头文件，用户在使用某种具体芯片进行开发时，需要在这里选择包含具体芯片头文件，如果没有定义，则提示错误信息：Please select first the target STM32F1xx device used in your application (in stm32f1xx.h file)。

```
#if defined(STM32F100xB)
  #include "stm32f100xb.h"
#elif defined(STM32F100xE)
  #include "stm32f100xe.h"
#elif defined(STM32F101x6)
  #include "stm32f101x6.h"
#elif defined(STM32F101xB)
  #include "stm32f101xb.h"
#elif defined(STM32F101xE)
  #include "stm32f101xe.h"
#elif defined(STM32F101xG)
  #include "stm32f101xg.h"
#elif defined(STM32F102x6)
  #include "stm32f102x6.h"
#elif defined(STM32F102xB)
  #include "stm32f102xb.h"
```



```
#elif defined(STM32F103x6)
  #include "stm32f103x6.h"
#elif defined(STM32F103xB)
  #include "stm32f103xb.h"
#elif defined(STM32F103xE)
  #include "stm32f103xe.h"
#elif defined(STM32F103xG)
  #include "stm32f103xg.h"
#elif defined(STM32F105xC)
  #include "stm32f105xc.h"
#elif
```

# 4.3.1 stm32f1xx.h源码分析

解析：为枚举类型enum{RESET=0, SET = !RESET}定义了两个别名FlagStatus和ITStatus，FlagStatus为标志状态,其值有两种RESET=0(重置）和SET =1(置位）；ITStatus为中断标志类型，其值有两种RESET = 0(重置）和SET =1(置位）。

```
typedef enum
{
  RESET = 0,
  SET = !RESET
} FlagStatus, ITStatus;

typedef enum
{
  DISABLE = 0,
  ENABLE = !DISABLE
} FunctionalState;

#define IS_FUNCTIONAL_STATE(STATE) (((STATE) == DISABLE) || ((STATE) == ENABLE))
```

解析：FunctionalState为函数状态，有两种状态，分别是DISABLE = 0(失能），ENABLE = 1(使能）。

解析：带参数的宏定义，用于功能状态参数判断，在函数中调用它来判断所输入的状态参数是不是ENABLE(使能）或DISABLE(失能），如果是则返回1，如果参数不合法则返回0。

# 4.3.1 stm32f1xx.h源码分析

```
typedef enum
{
  SUCCESS = 0U,
  ERROR = !SUCCESS
} ErrorStatus;
```

解析：ErrorStatus错误状态，SUCCESS = 0(正确），ERROR =1(错误）。

```
#define SET_BIT(REG, BIT)     ((REG) |= (BIT))
#define CLEAR_BIT(REG, BIT)   ((REG) &= ~(BIT))
#define READ_BIT(REG, BIT)    ((REG) & (BIT))
#define CLEAR_REG(REG)        ((REG) = (0x0))
#define WRITE_REG(REG, VAL)   ((REG) = (VAL))
#define READ_REG(REG)         ((REG))
#define MODIFY_REG(REG, CLEARMASK,
SETMASK)  WRITE_REG((REG),
(((READ_REG(REG)) & (~(CLEARMASK))) |
(SETMASK)))
#define POSITION_VAL(VAL)
(__CLZ(__RBIT(VAL)))
```

解析：STM32寄存器位的宏定义，其作用相当于函数。

解析：如果使用HAL驱动函数库，则应用程序中要include一个stm32f1xx_hal.h头文件。

```
#if defined (USE_HAL_DRIVER)
#include "stm32f1xx_hal.h"
#endif /* USE_HAL_DRIVER */

#ifdef __cplusplus
}
#endif /* __cplusplus*/
#endif /* __STM32F1xx_H */
```

# 4.3.2 stm32f103xe.h

stm32f103xe.h是STM32F1系列专门针对STM32F103xE芯片定义的特有的片上外设头文件。它包含了整个HAL库所共用的配置(中断向量、寄存器的定义等），也是所有外设与用户函数的头文件所必须包含的一个头文件。

- STM32F10x系列MCU的中断向量的定义
- 存储空间的地址映射
- 所有片上外设寄存器的结构体定义

# 4.3.2 stm32f103xe.h

```
#ifndef __STM32F103xE_H
#define __STM32F103xE_H

#ifdef __cplusplus
extern "C" {
#endif
```

解析：条件编译，防止头文件被重复包含。

解析：条件编译，类似if-else结构。当用C++编译器编译时，extern "C" { 告知编译器下面这段程序是用C语言编写的，请用C的方式进行编译。

```
#define __CM3_REV    0x0200U
      /*!< Core Revision r2p0           */
#define __MPU_PRESENT    0U
      /*!< Other STM32 devices does not provide an MPU  */
#define __NVIC_PRIO_BITS  4U
      /*!< STM32 uses 4 Bits for the Priority Levels    */
#define __Vendor_SysTickConfig    0U
       /*!< Set to 1 if different SysTick Config is used */
```

解析：与Cortex-M3内核相关的宏定义。

# 4.3.2 stm32f103xe.h

## MCU中断向量定义

解析：与Cortex-M3内核有关的异常中断，由ARM公司统一定义，编号均小于0，编号越小，意味着中断优先级越高。ARM公司设计的Cortex-M3内核可支持256个中断，包括15个内核中断和240个外部中断，并具有256级的可编程中断优先级设置，即除了3个固定的高优先级(Reset、NMI、硬件失效），其他中断和异常的优先级是可以由用户设置的。但使用Cortex-M3内核的芯片制造商(如ST、NXP公司等）往往不需要用到那么多中断，ST根据需求针对具体的MCU进行了裁剪。

```
    /*!< Interrupt Number Definition */
typedef enum
{
    /***  Cortex-M3 Processor Exceptions Numbers
***/
  NonMaskableInt_IRQn   = -14,
    /*!< 2 Non Maskable Interrupt                */
  HardFault_IRQn        = -13,
    /*!< 3 Cortex-M3 Hard Fault Interrupt        */
  MemoryManagement_IRQn= -12,
    /*!< 4 Cortex-M3 Memory Management Interrupt */
  BusFault_IRQn         = -11,
    /*!< 5 Cortex-M3 Bus Fault Interrupt         */
  UsageFault_IRQn       = -10,
    /*!< 6 Cortex-M3 Usage Fault Interrupt       */
  SVCall_IRQn           = -5,
    /*!< 11 Cortex-M3 SV Call Interrupt          */
  DebugMonitor_IRQn     = -4,
    /*!< 12 Cortex-M3 Debug Monitor Interrupt    */
  PendSV_IRQn           = -2,
    /*!< 14 Cortex-M3 Pend SV Interrupt          */
  SysTick_IRQn          = -1,
    /*!< 15 Cortex-M3 System Tick Interrupt      */
```

# 4.3.2 stm32f103xe.h

## MCU中断向量定义

解析：STM32的中断向量表，这个是由ST公司针对具体的STM32芯片设计的中断，STM32F103系列芯片有60个可屏蔽中断，中断向量编号为0~59。

```
    /**  STM32 specific Interrupt Numbers ******/
  WWDG_IRQn     = 0,
     /*!< Window WatchDog Interrupt          */
  PVD_IRQn       = 1,
     /*!< PVD through EXTI Line detection Interrupt  */
  TAMPER_IRQn    = 2,      /*!< Tamper Interrupt
*/
  RTC_IRQn                = 3,      /*!< RTC global Interrupt
*/
  FLASH_IRQn               = 4,      /*!< FLASH global Interrupt
*/
  RCC_IRQn                = 5,      /*!< RCC global Interrupt
*/
  EXTI0_IRQn               = 6,      /*!< EXTI Line0 Interrupt
*/
  EXTI1_IRQn               = 7,      /*!< EXTI Line1 Interrupt
*/

    ..... .....    ..

  DMA2_Channel3_IRQn     = 58,
```

# 4.3.2 stm32f103xe.h

# 所有片上外设寄存器的结构体定义

```
#include "core_cm3.h"
#include
"system_stm32f1xx.h"
#include <stdint.h>

typedef struct
{
  __IO uint32_t SR;
  __IO uint32_t CR1;
  __IO uint32_t CR2;
  __IO uint32_t SMPR1;
  __IO uint32_t SMPR2;
  __IO uint32_t JOFR1;
  __IO uint32_t JOFR2;
  __IO uint32_t JOFR3;
  __IO uint32_t JOFR4;
  __IO uint32_t HTR;
  __IO uint32_t LTR;
  __IO uint32_t SQR1;
  __IO uint32_t SQR2;
  __IO uint32_t SQR3;
  __IO uint32_t JSQR;
  __IO uint32_t JDR1;
  __IO uint32_t JDR2;
  __IO uint32_t JDR3;
  __IO uint32_t JDR4;
  __IO uint32_t DR;
} ADC_TypeDef;
……
typedef struct
{
  __IO uint32_t CR;
     /*!< WWDG Control register,       Address offset: 0x00 */
  __IO uint32_t CFR;
    /*!< WWDG Configuration register, Address offset: 0x04
*/
  __IO uint32_t SR;
    /*!< WWDG Status register,       Address offset: 0x08 */
} WWDG_TypeDef;
```

解析：STM32F103xE芯片的片上外设寄存器的结构体定义。

# 4.3.2 stm32f103xe.h

## 存储空间的地址映射

解析：应用程序的编写是通过调用HAL库的接口函数实现相应的功能，其实质是调用标准外设库函数来完成相应寄存器的操作。

地址利用指针转换为寄存器，：通过宏定义#define GPIOB（(GPIO_TypeDef *)GPIOB_BASE)，转换为GPIO结构体的指针，这样就可以操作结构体的成员(也即寄存器）。

```
#define FLASH_BASE        0x08000000UL  /*FLASH 基地址*/
#define FLASH_BANK1_END  0x0807FFFFUL  /* FLASH bank1结束地址 */
#define SRAM_BASE         0x20000000UL  /*SRAM 基地址 */
#define PERIPH_BASE       0x40000000UL /*外设基地址*/
   /*!外设总线地址*/
#define APB1PERIPH_BASE       PERIPH_BASE
#define APB2PERIPH_BASE       (PERIPH_BASE + 0x00010000UL)
#define AHBPERIPH_BASE        (PERIPH_BASE + 0x00020000UL)

#define TIM2_BASE      (APB1PERIPH_BASE + 0x00000000UL)
……
#define GPIOA_BASE     (APB2PERIPH_BASE + 0x00000800UL)
……
#define ADC1_BASE      (APB2PERIPH_BASE + 0x00002400UL)
……
#define USART1_BASE          (APB2PERIPH_BASE + 0x00003800UL)

#define TIM2            ((TIM_TypeDef *)TIM2_BASE)
.. …
#define GPIOA           ((GPIO_TypeDef *)GPIOA_BASE)
……
#define DMA1_Channel2   ((DMA_Channel_TypeDef
```

# 4.3.2 stm32f103xe.h

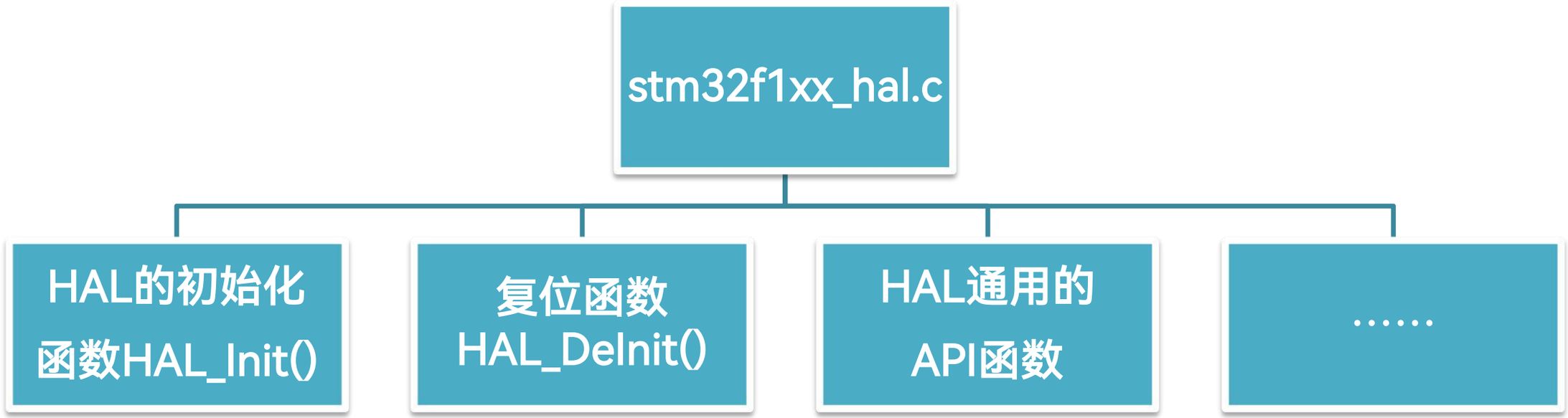
## 外设寄存器位定义

```
#define GPIO_CRL_MODE_Pos     (0U)
#define GPIO_CRL_MODE_Msk     (0x33333333UL <<
GPIO_CRL_MODE_Pos)
    /*!< 0x33333333 */
#define GPIO_CRL_MODE       GPIO_CRL_MODE_Msk
    /*!< Port x mode bits */
… …
```

解析：stm32f103xe.h文件中包含了片上外设的寄存器位定义，比如：CRC、GPIO、EXTI、DMA、ADC、DAC、TIM、IIC、SPI、CAN、USB、SDIO、FSMC等。

# 4.3.3 stm32f1xx_hal.c和stm32f1xx_hal.h

stm32f1xx_hal.c和stm32f1xx_hal.h是STM32F1系列MCU的HAL库相关的源文件和头文件。



# 4.3.3 stm32f1xx_hal.c和stm32f1xx_hal.h

```
  /* ----包含的头文件--*/
#include  "stm32f1xx_hal.h"
… …
__IO uint32_t  uwTick;
     //定义一个32位的无符号整型变量uwTick
uint32_t  uwTickPrio   = (1UL << __NVIC_PRIO_BITS);
     /* 无效的优先级PRIO */
HAL_TickFreqTypeDef   uwTickFreq =
HAL_TICK_FREQ_DEFAULT;
     /* 1KHz */
```

HAL_TICK_FREQ_DEFAULT定义在stm32f1xx_hal.h头文件中，是别名为HAL_TickFreqTypeDef的一个枚举成员，其值为1ms:

解析：定义滴答定时器的时基，HAL_TICK_FREQ_1KHZ为1ms，HAL_TICK_FREQ_100HZ为10ms，HAL_TICK_FREQ_10HZ为100ms。

```
typedef enum
{
  HAL_TICK_FREQ_10HZ  = 100U,
  HAL_TICK_FREQ_100HZ  = 10U,
  HAL_TICK_FREQ_1KHZ   = 1U,
  HAL_TICK_FREQ_DEFAULT  =
AL_TICK_FREQ_1KHZ
} HAL_TickFreqTypeDef;
```

# 4.3.3 stm32f1xx_hal.c和stm32f1xx_hal.h

```
HAL_StatusTypeDef HAL_Init(void)
{
#if (PREFETCH_ENABLE != 0)
#if defined(STM32F101x6) || defined(STM32F101xB)
|| defined(STM32F101xE) || defined(STM32F101xG) ||
\
defined(STM32F102x6) || defined(STM32F102xB) || \
defined(STM32F103x6) || defined(STM32F103xB) ||
defined(STM32F103xE) || defined(STM32F103xG) || \
defined(STM32F105xC) || defined(STM32F107xC)
  __HAL_FLASH_PREFETCH_BUFFER_ENABLE();
#endif
#endif /* PREFETCH_ENABLE */

HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGRO
UP_4);
  HAL_InitTick(TICK_INT_PRIORITY);
  HAL_MspInit();
return HAL_OK;
```

解析：HAL的初始化函数HAL_Init()，通过配置Flash预取功能实现指令和数据的预取缓存。

通过HAL_NVIC_SetPriorityGrouping( )函数设置中断优先级分组为NVIC_PRIORITYGROUP_4，调用HAL_InitTick()函数配置系统滴答定时器SysTick产生1ms的时间基准，调用HAL_MspInit回调函数，执行外设驱动的初始化(如GPIO、串口、DMA等），从而完成系统初始化。

# 4.3.3 stm32f1xx_hal.c和stm32f1xx_hal.h

```
__weak void HAL_MspInit(void)
{
  /* NOTE : This function should not be modified,
when the callback is needed,
the HAL_MspInit could be implemented in the user file
   */
}
```

解析：HAL_MspInit( )为回调函数，函数体内部是空的，由程序开发人员完成。注意：函数有个前缀__weak，表示弱定义，用户可以重定义。

```
__weak HAL_StatusTypeDef
HAL_InitTick(uint32_t TickPriority)
{
    /*配置系统滴答定时器产生时基为1ms的中断*/
if (HAL_SYSTICK_Config(SystemCoreClock /
(1000U / uwTickFreq)) > 0U)
  {
return HAL_ERROR;
  }
```

```
    /*配置系统滴答定时器SysTick的中断优先级*/
if (TickPriority < (1UL << __NVIC_PRIO_BITS))
  {
    HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority,
0U);
    uwTickPrio = TickPriority;
  }
else
  {    return HAL_ERROR;  }
return HAL_OK;
}
```

# 4.3.3 stm32f1xx_hal.c和stm32f1xx_hal.h

```
__weak  void  HAL_IncTick(void)
{    uwTick += uwTickFreq;
}
```

解析：调用了函数 HAL_Init 后，SysTick 相关的函数就可以使用

```
__weak  uint32_t  HAL_GetTick(void)
{   return uwTick;
}
```

解析：HAL_GetTic( )函数用于获取计数器uwTick的值。

```
__weak  void  HAL_Delay(uint32_t Delay)
{
uint32_t tickstart = HAL_GetTick();
uint32_t wait = Delay;
if (wait < HAL_MAX_DELAY)
  {    wait += (uint32_t)(uwTickFreq);
  }
while ((HAL_GetTick() - tickstart) < wait)
  {    }
}
```

解析：HAL_Delay( )延时函数通过循环读取uwTick 计数器的值与输入Delay的值相比较，根据差值来实现延迟功能，输入参数Delay为需要延时的时间，单位为毫秒(ms)

HAL_Delay( )延时函数是在应用程序中调用较多的一个函数。

# 4.3.4 stm32f1xx_hal_gpio.c 和stm32f1xx_hal_gpio

## stm32f1xx_hal_xxx.h

某一个具体外设的头文件，主要包含具体外设功能的结构体的定义以及外设HAL库函数的声明。

## stm32f1xx_hal_xxx.c

某一个功能外设硬件驱动HAL库函数的源文件，是xxx外设所用HAL库函数的具体实现。

# 4.3.4 stm32f1xx_hal_gpio.c 和stm32f1xx_hal_gpio

```
  /* Define to prevent recursive inclusion
---*/
#ifndef STM32F1xx_HAL_GPIO_H
#define STM32F1xx_HAL_GPIO_H
#ifdef __cplusplus
extern "C" {
#endif
#include "stm32f1xx_hal_def.h “
typedef struct
{   uint32_t Pin;
    uint32_t Mode;
    uint32_t Pull;
    uint32_t Speed;
} GPIO_InitTypeDef;
typedef enum
{  GPIO_PIN_RESET = 0u,
    GPIO_PIN_SET
} GPIO_PinState;
```

解析：定义一个struct结构体的别名GPIO_InitTypeDef，其成员变量有四个，Pin代表引脚，Mode表示输入输出模式，Pull表示上拉下拉，Speed表示速度。

解析：定义一个enum枚举类型的别名GPIO_PinState，用于表示GPIO引脚置位和复位的定义，GPIO_PIN_RESET表示GPIO引脚复位，即置“0”；GPIO_PIN_SET表示GPIO引脚置位，即置“1”。

# 4.3.4 stm32f1xx_hal_gpio.c 和stm32f1xx_hal_gpio

```
  /* Define to prevent recursive inclusion ---*/
#define GPIO_PIN_0     ((uint16_t)0x0001)
    /* Pin 0 selected    */
#define GPIO_PIN_1     ((uint16_t)0x0002)
    /* Pin 1 selected    */
…….
 #define GPIO_PIN_15    ((uint16_t)0x8000)
    /* Pin 15 selected   */
#define GPIO_PIN_All    ((uint16_t)0xFFFF)
   /* All pins selected */
#define GPIO_PIN_MASK    0x0000FFFFu
  /* PIN mask for assert test */
```

解析：GPIO引脚定义。

```
#define  GPIO_MODE_INPUT     0x00000000u
      /*输入模式 */
#define  GPIO_MODE_OUTPUT_PP      0x00000001u
      /*推挽输出模式  */
#define  GPIO_MODE_OUTPUT_OD      0x00000011u
      /*开漏输出模式  */
#define  GPIO_MODE_AF_PP          0x00000002u
      /*复用推挽输出模式  */
#define  GPIO_MODE_AF_OD          0x00000012u
      /*复用开漏输出模式   */
#define  GPIO_MODE_AF_INPUT
GPIO_MODE_INPUT
      /*复用输入模式 */
… …
#define  GPIO_MODE_EVT_RISING_FALLING
0x10320000u
```

解析：GPIO引脚输入输出模式定义，输出模式主要有推挽、开漏模式。

# 4.3.4 stm32f1xx_hal_gpio.c 和stm32f1xx_hal_gpio

```
  /* Define to prevent recursive inclusion ---*/
#define  GPIO_SPEED_FREQ_LOW
(GPIO_CRL_MODE0_1)
  /*!< Low speed */
#define  GPIO_SPEED_FREQ_MEDIUM
(GPIO_CRL_MODE0_0)
  /*!< Medium speed */
#define  GPIO_SPEED_FREQ_HIGH    (GPIO_CRL_MODE0)
  /*!< High speed */
```

解析：GPIO引脚输出速度的定义，有三种：低速、中速、高速，这里的输出速度指的是I/O引脚的响应速度。

stm32f1xx_hal_gpio.h头文件中除了相应的宏定义，源码最后一部分是GPIO HAL库的函数声明。

```
/*--- GPIO初始化及复位函数---*/
void  HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *GPIO_Init);
void  HAL_GPIO_DeInit(GPIO_TypeDef  *GPIOx, uint32_t GPIO_Pin);
/* ---GPIO引脚功能操作函数---*/
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin,
GPIO_PinState PinState);
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
HAL_StatusTypeDef HAL_GPIO_LockPin(GPIO_TypeDef *GPIOx, uint16_t
GPIO_Pin);
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin);
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);
```

# 本章小结

## 4.1 STM32最小系统

- 电源电路
- 时钟电路
- 复位电路
- 调试和下载电路
- LED指示电路

## 4.2 嵌入式C语言

- 数据类型
- 各关键词
- 指针
- 回调函数

## 4.3 HAL库源码分析

- stm32f1xx.h
- stm32f103xe.h
- stm32f1xx_hal.c
- stm32f1xx_hal.h
- stm32f1xx_hal_gpio.c
- stm32f1xx_hal_gpio.h




# The End!

